



Most developers learn Spring Boot by copying tutorials. Few understand why it exists.

Spring Boot

Complete Beginner Guide



Setup



Build



Database



Production

“ Spring Boot exists because configuring Java applications manually became painful.

✂ THE PROBLEM

What **Spring Boot** Solves

✖ Before

- 📄 XML configuration
- 🔌 Server setup
- 🔌 Dependency setup

✓ After

- 🔌 Auto configuration
- 📦 Embedded server
- ✈ Faster development

📌 Spring Boot simplifies Spring.



Every Spring Boot application starts with four tools.



SETUP

System Prerequisites



Java

Runtime Environment



IDE

Development (IntelliJ / Eclipse)



Build Tool

Packaging (Maven / Gradle)



Git

Version Control

 Spring Boot supports both. Choose intentionally.

BUILD SYSTEMS

Maven vs Gradle

Maven

 XML Configuration

 Highly Stable

 Beginner friendly

Gradle

 Much Faster

 Highly Flexible

 Large-scale builds

Start with Maven. Learn Gradle later.

 Your first application should take less than a minute.

▶ GETTING STARTED

Create Your **First Project**

 Go to: start.spring.io



☒ Select: Java, Maven, Jar



 Add: Spring Web



 Generate

 Folders become easy once you know their purpose.

ARCHITECTURE

Understanding **Structure**



src/main/java

Your Application Code



resources

Configuration files (application.yml)



pom.xml

Maven Dependencies



target

Build output & compiled files



A Spring Boot app starts from one annotation.



EXECUTION

Run Your **Application**

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class,
args);
    }
}
```

```
>_ mvn spring-boot:run
```

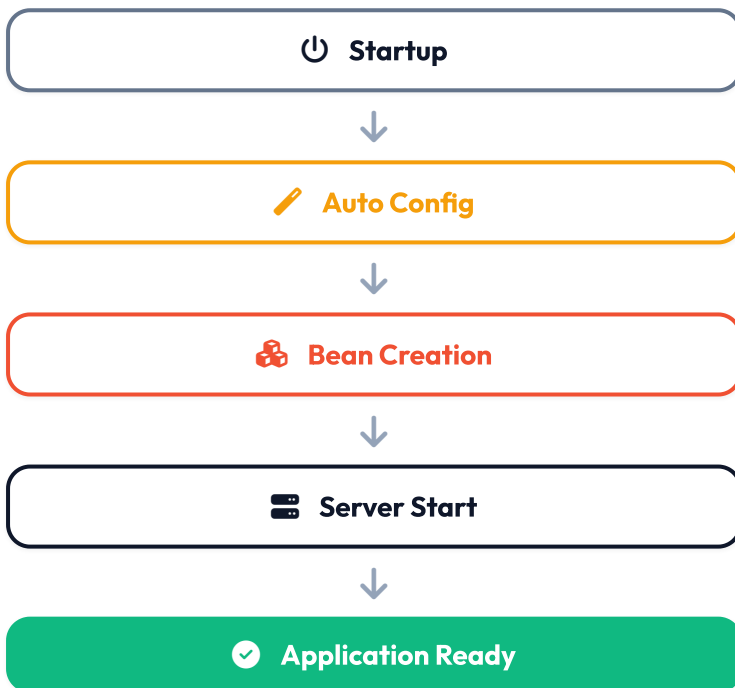


Default URL: localhost:8080

🔄 More happens than just executing `main()`.

⚙️ INTERNALS

Spring Boot Lifecycle





Senior engineers create fewer objects manually.



CORE CONCEPT

Dependency Injection



Bad Approach

```
Service service = new Service();
```



Better Approach

Spring manages objects (Beans)

```
private final Service service;  
public Controller(Service service) {  
    this.service = service;  
}
```

Prefer Constructor Injection over @Autowired



Modern applications are assembled, not written from scratch.



ECOSYSTEM

Why Dependencies **Matter**



spring-boot-starter-web

Unlocks REST APIs & embedded Tomcat



spring-boot-starter-data-jpa

Unlocks database operations & Hibernate



spring-boot-starter-security

Unlocks authentication & authorization



Dependencies unlock core capabilities.

➡ Controllers are the entry point of every API.

🔗 API LAYER

Building Controllers

```
@RestController
public class ApiController {

    @GetMapping("/data")
    public Response getData() { ... }

    @PostMapping("/data")
    public Response addData() { ... }
}
```

📱 Client Request



☰ API Controller

⚠ Business logic should never live inside controllers.

📁 BUSINESS LOGIC

The Service Layer

```
@Service
public class UserService {
    public User processUser(User data) {
        // 1. Validate data
        // 2. Apply business rules
        // 3. Save via repository
    }
}
```

☰ Responsibilities

- ✓ Data Validation
- ✓ Business Rules Enforcement
- ✓ Data Processing



Databases should not leak into business logic.



DATA ACCESS

The Repository Layer

```
@Repository
public interface UserRepository
    extends JpaRepository<User, Long> {

    // Custom query methods if needed
    List<User> findByEmail(String email);
}
```



JpaRepository Magic

Provides full CRUD operations without writing any SQL queries manually.

 Applications become useful when data survives restarts.

 STORAGE

Database **Integration**

 PostgreSQL

 MySQL

Configure in `application.yml`

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/db
    username: admin
    password: secret
  jpa:
    hibernate:
      ddl-auto: update
```

 Spring Boot handles all connections.

 Production systems separate config from code.

 ENVIRONMENTS

Configuration Management

 `application.yml`

PROFILES

dev

test

prod

 Never hardcode environments in code.

 Trusting user input creates bugs.

SECURITY

Input Validation

```
public class UserDto {  
  
    @NotNull  
    private String name;  
  
    @Email  
    private String email;  
}  
  
// In Controller:  
public Response create(@Valid @RequestBody UserDto  
dto)
```

 **Validate everything at the boundaries.**



Production systems fail gracefully.

! ROBUSTNESS

Exception Handling

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(NotFoundException.class)
    public ResponseEntity<ErrorDto>
handleNotFound() {
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(new ErrorDto("Not found"));
    }
}
```



Useful Messages



Correct Status Codes

🔍 Production debugging starts with observability.

📈 OPERATIONS

Logging & Monitoring



Structured Logging



Application Metrics



Health Checks



Explore: Spring Boot Actuator

 Code that works locally is not tested.

✓ QUALITY

Automated Testing

```
@SpringBootTest
class ApplicationTests {

    @Test
    void contextLoads() {
        // Validates context starts
    }
}
```

 JUnit

 MockMvc

Automate confidence in your deployments.



You now know enough to start building.



ROADMAP

What To Learn Next



Spring Security



OAuth2



Okta



Redis



Kafka



Docker



Microservices



Cloud Deployment

Build projects.

That is where Spring Boot starts making sense.